

第5章 数组和广义表 (Arrays & Lists)

数组和广义表的特点：一种特殊的线性表

- ① 元素的值并非原子类型，可以再分解，表中元素也是一个线性表（即广义的线性表）。
- ② 所有数据元素仍属**同一数据类型**。

5.1 数组的定义

5.2 数组的顺序表示和实现

5.3 矩阵的压缩存储

5.4 广义表的定义

5.5 广义表的存储结构

5.1 数组的定义

数组： 由一组名字相同、下标不同的变量构成

注意： 本章所讨论的数组与高级语言中的数组有所区别：高级语言中的数组是顺序结构；而**本章的数组既可以是顺序的，也可以是链式结构**，用户可根据需要选择。

讨论： “数组的处理比其它复杂的结构要简单”，对吗？

答： **对的。** 因为：

- ① 数组中各元素具有**统一的类型**；
- ② 数组元素的下标一般具有**固定的上界和下界**，即数组一旦被定义，它的维数和维界就不再改变。
- ③ 数组的**基本操作比较简单**，除了结构的初始化和销毁之外，只有存取元素和修改元素值的操作。

一维数组的特点： 1个下标， a_i 是 a_{i+1} 的直接前驱

二维数组的特点： 2个下标，每个元素 $a_{i,j}$ 受到两个关系
(行关系和列关系) 的约束：

$$A_{mn} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

一个 $m \times n$ 的二维数组可以看成是 m 行的一维数组，或者 n 列的一维数组。

N维数组的特点： n 个下标，每个元素受到 n 个关系约束

一个 n 维数组可以看成是由若干个 $n - 1$ 维数组组成的线性表。

N 维数组的数据类型定义

$n_ARRAY = (D, R)$ 其中:

数据对象: $D = \{a_{j_1, j_2, \dots, j_n} \mid j_i \text{ 为数组元素的第 } i \text{ 维下标}, a_{j_1, j_2, \dots, j_n} \in \text{Elemset}\}$
数据关系: $R = \{R_1, R_2, \dots, R_n\}$

$R_i = \{ \langle a_{j_1, j_2, \dots, j_i, \dots, j_n}, a_{j_1, j_2, \dots, j_{i+1}, \dots, j_n} \rangle \mid$

$a_{j_1, j_2, \dots, j_i, \dots, j_n}, a_{j_1, j_2, \dots, j_{i+1}, \dots, j_n} \in D, i=2, \dots,$

$n\}$

基本操作: 构造数组、销毁数组、读数组元素、写数组元素

数组的抽象数据类型定义略, 参见教材 P90

5.2 数组的顺序存储表示和实现

问题：计算机的存储结构是一维的，而数组一般是多维的，怎样存放？

解决办法：事先约定按某种次序将数组元素排成一系列序列，然后将这个线性序列存入存储器中。

例如：在二维数组中，我们既可以规定按**行**存储，也可以规定按**列**存储。

注意：

- 若规定好了次序，则数组中任意一个元素的存放地址便有规律可寻，可形成地址计算公式；
- 约定的次序不同，则计算元素地址的公式也有所不同；
- **C**和 **PASCAL** 中一般采用行优先顺序； **FORTAN**采用列优先。

无论规定行优先或列优先，只要知道以下三要素便可随时求出任一元素的地址（这样数组中的任一元素便可以随机存取！）：

- ①开始结点的存放地址（即基地址）
- ②维数和每维的上、下界；
- ③每个数组元素所占用的单元数

$$A_{mn} = \begin{bmatrix} a_{c_1, c_2} & \cdots & a_{c_1, d_2} \\ \cdots & a_{ij} & \cdots \\ a_{d_1, c_2} & \cdots & a_{d_1, d_2} \end{bmatrix}$$

补充：计算二维数组元素地址的通式

设一般的二维数组是 $A[c_1..d_1, c_2..d_2]$ ，这里 c_1, c_2 不一定是 0。

则行优先存储时的地址公式为：

$$LOC(a_{ij}) = LOC(a_{c_1, c_2}) + [(i - c_1) * (d_2 - c_2 + 1) + j - c_2] * L$$

数组基址

a_{ij} 之前的
行数

总列数 即
第 2 维长度

a_{ij} 本行前面
的元素个数

单个元素
长度

二维数组列优先存储的通式为：

$$LOC(a_{ij}) = LOC(a_{c_1, c_2}) + [(j - c_2) * (d_1 - c_1 + 1) + i - c_1] * L$$

例 1 【软考题】：一个二维数组 A，行下标的范围是 1 到 6，列下标的范围是 0 到 7，每个数组元素用相邻的 6 个字节存储，存储器按字节编址。那么，这个数组的体积是_____个字节。

答： $\text{Volume} = m * n * L = (6 - 1 + 1) * (7 - 0 + 1) * 6 = 48 * 6 = 288$

例 2：已知二维数组 $A_{m,m}$ 按行存储的元素地址公式是：

$\text{Loc}(a_{ij}) = \text{Loc}(a_{11}) + [(i-1)*m + (j-1)] * K$ ，按列存储的公式是？

$\text{Loc}(a_{ij}) = \text{Loc}(a_{11}) + [(j-1)*m + (i-1)] * K$ （尽管是方阵，但公式仍不同）

例 3：【00 年计算机系考研题】设数组 $a[1...60, 1...70]$ 的基地址为 2048，每个元素占 2 个存储单元，若以列序为主序顺序存储，则元素 $a[32, 58]$ 的存储地址为_____

答：请注意审题！ 利用列优先通式：

$$\text{LOC}(a_{ij}) = \text{LOC}(a_{c_1, c_2}) + [(j - c_2) * (d_1 - c_1 + 1) + i - c_1] * L$$

$$\text{得： } \text{LOC}(a_{32, 58}) = 2048 + [(58 - 1) * (60 - 1 + 1) + 32 - 1] * 2 = 8950$$

若是 N 维数组，其中任一元素的地址该如何计算？

教材已给出低维优先的地址计算公式，见 P93 (5-2) 式

该式称为 n 维数组的映像函数：

$$\text{Loc}(j_1, j_2, \dots, j_n) = \text{LOC}(0, 0, \dots, 0) + \sum_{i=1}^n C_i j_i$$

数组基址

前面若干元素占用的地址字节总数

其中 $C_n = L$, $C_{i-1} = b_i \times C_i$, $1 < i \leq n$

一个元素长度

第 i 维长度

与所存元素个数有关的系数，可用递推法求出

N 维数组的顺序存储表示 (见教材 P93)

```
#define MAX_ARRAY_DIM 8 // 假设最大维数为 8

typedef struct{
    ELemType *base;      // 数组元素基址
    int      dim;         // 数组维数
    int      *bound;      // 数组各维长度信息保存区基址
    int      *constants; // 数组映像函数常量的基址
}Array;
```

即 C_i 信息保存

区

数组的基本操作函数说明 (有 5 个)

(请阅读教材 P93-95)

以销毁数组函数为例

数组的基本操作函数说明（5个）（见教材 P93-95）

```
Status InitArray(Array &A,int dim,...){
```

```
// 若维数 dim 和各维长度合法，则构造相应的数组 A 并返回 OK
```

```
if(dim<1||dim>MAX_ARRAY_DIM) return ERROR;
```

```
A.dim=dim;
```

```
A.bounds=(int *)malloc(dim * sizeof(int));
```

```
if(!a.bounds) exit(OVERFLOW);
```

```
// 若各维长度合法，则存入 A bounds，并求出 A 的元素总数 elementotal
```

```
elemtotal=1;
```

```
va_start(ap,dim); //ap 为 va_list 类型，是存放变长参数表信息的类型
```

```
for(i=0;i<dim;++i){
    A.bounds[i]=va_arg(ap,int);
    if(A.bounds[i]<0) return UNDERFLOW;
    elemtotal *=A.bounds[i];  }
va_end(ap);
A.base=(ElemType * )malloc(elemtotal *
    sizeof(ElemType));
if(!A.base) exit(OVERFLOW);
A.constants=(int * )malloc(dim *sizeof(int));
if(!A.constants) exit(OVERFLOW);
A.constants[dim-1]=1;
for(i=dim-2;i>=0;--i)
    A.constants[i]=A.bounds[i+1]*A.constants[i+1];
return OK;
}
```

Status **DestroyArray**(Array &A)

{ **// 销毁数组 A**

if (! A . base) return ERROR;

free(A . base);

数组基址指针

A . base = NULL;

if (! A.bounds) return ERROR;

free(A . bounds);

各维长度保
存区指针

A . bounds = NULL;

if (!A.constants) return ERROR;

free (A. constants);

映像函数 G
保存区指针

A. constants = NULL;

return OK;

}

```
Status Locate(Array A,va_list ap,int &off)
```

```
{
```

```
// 若 ap 指示的各下标值合法，则求出该元素在 A 中，相对地址  
off
```

```
off=0;
```

```
for(i=0;i<A.dim;++i)
```

```
{
```

```
ind=va_arg(ap,int);
```

```
if(ind<0||ind>A.bounds[i]) return OVERFLOW;
```

```
off+=A.constants[i] *ind;
```

```
}
```

```
return OK;
```

```
}
```

```
Status Value(Array A,ElemType &e,...){
```

**//A是 n维数组, e为元素变量, 随后是 n个下标值,
若各下标不超界, 则 e赋值为所指定的 A的元素值, 即
将指定元素值读到 e变量中。**

```
    va_start(ap,e);
```

```
    if((result=Locate(A,ap,off))<=0) return result;
```

```
    e=*(A.base+off);
```

```
    return OK;
```

```
}
```

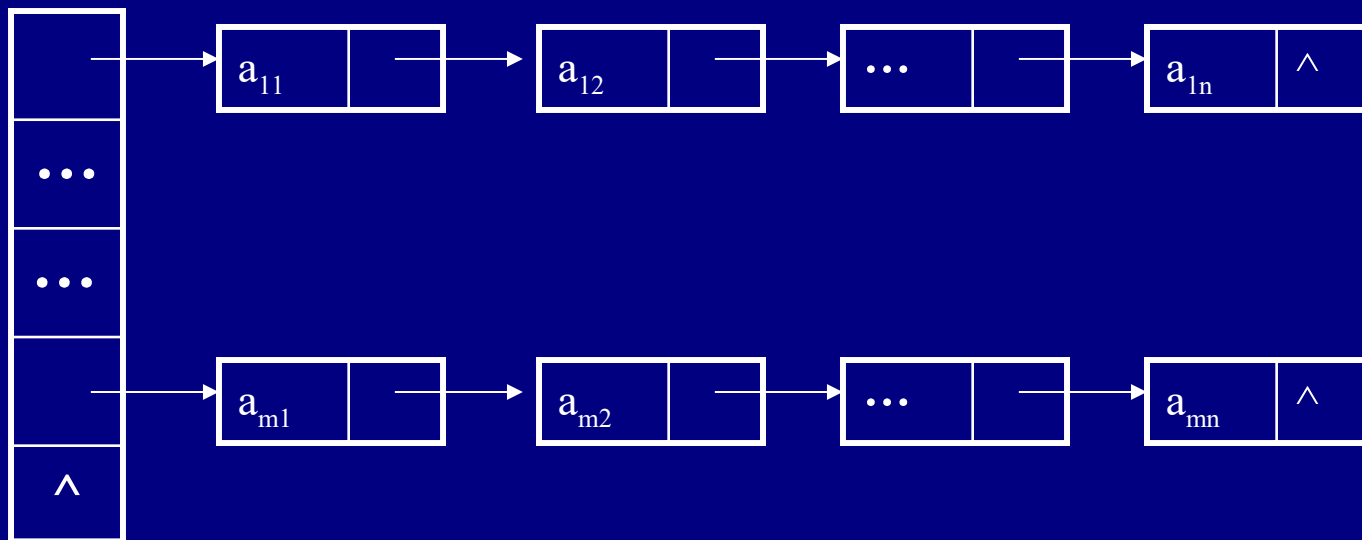
```
Status Assign(Array &A,ElemType e,...){  
    //A是 n维数组, e为元素变量, 随后是 n个下标  
    值, 若各下标不超界, 则 e的值赋为所指定的 A的  
    元素值, 即: 将 e值写入指定数组单元。  
    va_start(ap,e);  
    if((result=Locate(A,ap,off))<=0) return  
result;  
    *(A.base+off)=e;  
    return OK;  
}
```

顺序存储方式：按低地址优先（或高地址优先）顺序存入一维数组。
(难点是多维数组与一维数组的地址映射关系)

补充：

链式存储方式：用带行指针向量的单链表来表示。

行指针向量



注：数组的运算参见下一节实例（稀疏矩阵的转置）

5.3 矩阵的压缩存储

讨论：

1. 什么是压缩存储？

若多个数据元素的值都相同，则只分配一个元素值的存储空间，且零元素不占存储空间。

2. 所有二维数组（矩阵）都能压缩吗？

未必，要看矩阵是否具备以上压缩条件。

3. 什么样的矩阵具备以上压缩条件？

一些特殊矩阵，如：对称矩阵，对角矩阵，三角矩阵，稀疏矩阵等。

4. 什么叫稀疏矩阵？

矩阵中非零元素的个数较少（一般小于 5%）

重点介绍稀疏矩阵的压缩和相应的操作。

一、稀疏矩阵的压缩存储

二、稀疏矩阵的操作



问题:

如果只存储稀疏矩阵中的非零元素，那这些元素的位置信息该如何表示？

解决思路:

对每个非零元素增开若干存储单元，例如存放其所在的行号和列号，便可准确反映该元素所在位置。

实现方法:

将每个非零元素用一个三元组 (i, j, a_{ij}) 来表示，则每个稀疏矩阵可用一个三元组表来表示。

例 1 : 三元素组表中的每个结点对应于稀疏矩阵的一个非零元素，它包含有三个数据项，分别表示该元素的_____和_____。
 —、——行下标——和——列下标——元素值。

例 2 : 写出右图所示稀疏矩阵的压缩存储形式。

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

法 1 : 用线性表表示:

((1, 2, 12) , (1, 3, 9) , (3, 1, -3) , (3, 5, 14) ,
 (4, 3, 24) , (5, 2, 18) , (6, 1, 15) , (6, 4, -7))

法 2：用三元组矩阵表示：

	i	j	value
0	6	6	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	5	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

注意：为更可靠描述，通常再加一行“总体”信息：即**总行数、总列数、非零元素总个数**

稀疏矩阵压缩存储的缺点：将失去随机存取功能 :- (

法三：用带辅助向量的三元组表示。

用途：通过三元组**高效访问稀疏矩阵中任一非零元素。**

方法：增加 2 个辅助向量：

① **记录每行非 0 元素个数，用 $NUM(i)$ 表示；**

② **记录稀疏矩阵中每行第一个非 0 元素在**三元组中的行号**，用 $POS(i)$ 表示。**

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

	i	j	v
0	6	6	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	5	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

i	1	2	3	4	5	6
NUM(i)	2	0	2	1	1	2
POS(i)	1	3	3	5	6	7

规律： $POS(1) = 1$

$POS(i) = POS(i-1) + NUM(i-1)$

法四：用十字链表表示

用途：方便稀疏矩阵的加减运算；

方法：每个非 0 元素占用 5 个域。

i	j	v
down		right

同一列中下一非零元素的指针

同一行中下一非零元素的指针

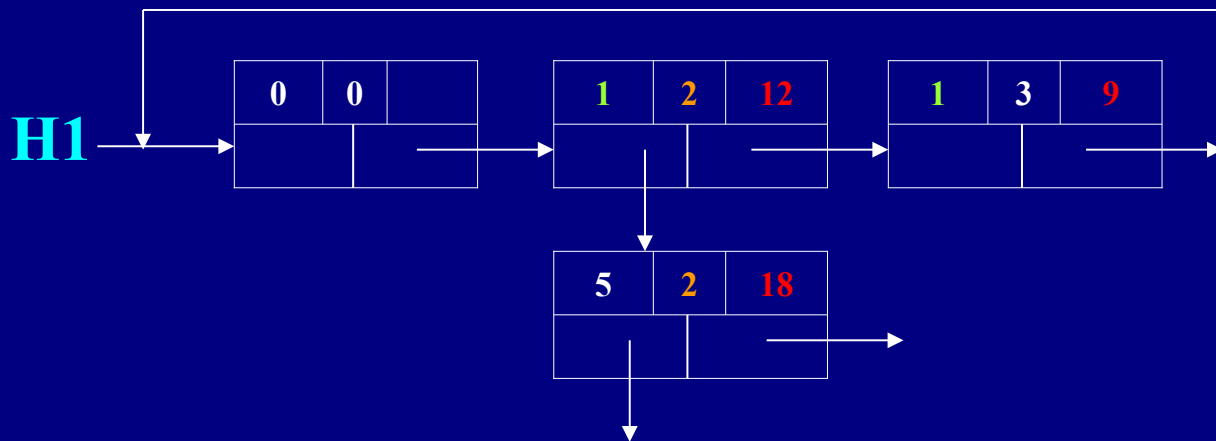
十字链表的特点：

①每行非零元素链接成带表头结点的循环链表；

②每列非零元素也链接成带表头结点的循环链表。

则每个非零元素既是行循环链表中的一个结点；又是列循环链表中的一个结点，即呈十字链状。

以刚才的
稀疏矩阵
为例：



三元组表的顺序存储表示（见教材 P98 ）：

```
#define MAXSIZE 125000    // 设非零元素最大个数 125000

typedef struct{
    int i;                // 元素行号
    int j;                // 元素列号
    ElemType e;           // 元素值
}Triple;                 // 一个结点的结构定义

typedef struct{
    Triple data[MAXSIZE+1]; // 三元组表，以行为主序存入一维向量
                                data[ ] 中
    int mu;                 // 矩阵总行数
    int nu;                 // 矩阵总列数
    int tu;                 // 矩阵中非零元素总个数
}TsMatrix;                // 整个三元组表的定义
```

二、稀疏矩阵的操作 (以转置运算为例)

目的:

$$M = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

转置后



$$T = \begin{pmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

T

三元组表
a.data

(1, 2, 12)
(1, 3, 9)
(3, 1, -3)
(3, 5, 14)
(4, 3, 24)
(5, 2, 18)
(6, 1, 15)
(6, 4, -7)

三元组表
b.data

(1, 3, -3)
(1, 6, 15)
(2, 1, 12)
(2, 5, 18)
(3, 1, 9)
(3, 4, 24)
(4, 6, -7)
(5, 3, 14)

提问：若采用三元组压缩技术存储稀疏矩阵，只要把每个元素的**行下标和列下标互换**，就完成了对该矩阵的转置运算，这种说法正确吗？

答：肯定不正确！

除了：（1）每个元素的行下标和列下标互换（即三元组中的**i 和 j 互换**）；

还应该：（2）T的总行数 **mu** 和总列数 **nu** 与 M值不同（互换）；

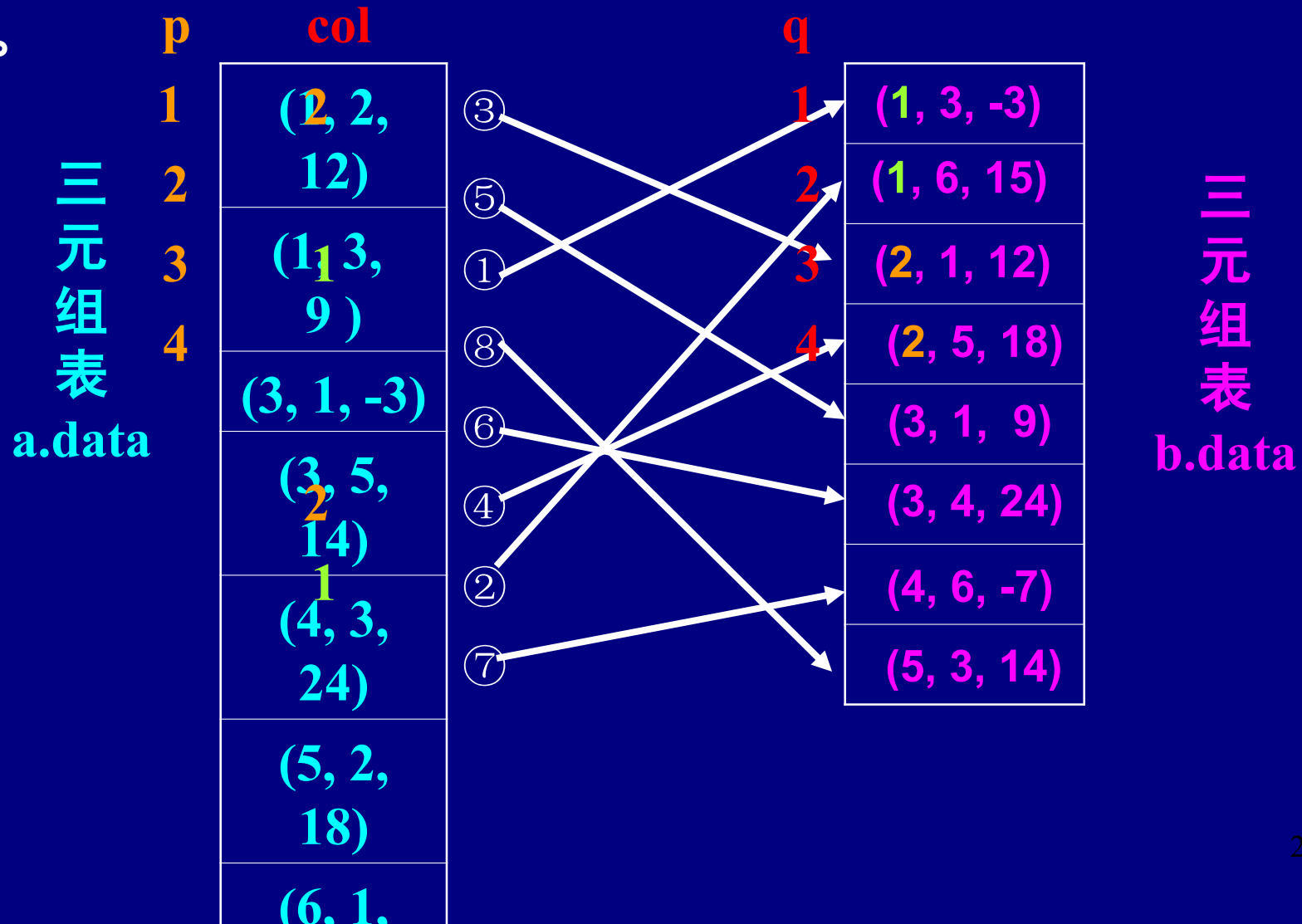
（3）**重排三元组内元素顺序**，使转置后的三元组也按行（或列）为主序有规律的排列。

上述（1）和（2）容易实现，难点在（3）。

有两种实现方法 { 压缩转置
（压缩）快速转置

方法 1：压缩转置

思路：反复扫描 a.data 中的列序，从小到大依次进行转置。



压缩转置算法描述：（见教材 P99）

```
Status TransPoseSMatrix(TSMatrix M, TSMatrix &T)
{
    // 用三元组表存放稀疏矩阵 M, 求 M 的转置矩阵 T
    T.mu=M.nu; T.nu=M.mu; T.tu=M.tu;

    if (T.tu) {
        q=1;           // q 是转置矩阵 T 的结点编号
        for(col=1; col<=M.nu; col++) // cd 是扫描 M 元表列序的变量
        {for(p=1; p<=M.tu; p++)      // p 是 M 元表中结点编号
            {if (M.data[p].j==col)
                {T.data[q].i=M.data[p].j; T.data[q].j=M.data[p].i;
                 T.data[q].value=M.data[p].value; q++;
                }
            }
        }
    }
    return OK;
} //TranposeSMatrix;
```

压缩转置算法的效率分析:

1、主要时间消耗在查找 $Mdata[p].j = col$ 的元素, 由两重循环完成:

```
for(col=1; col<=M.nu; col++) 循环次数 = nu
    {for(p=1; p<=M.tu; p++) 循环次数 = tu}
```

所以该算法的时间复杂度为 $O(nu*tu)$

---- 即 M 的列数与 M 中非零元素的个数之积

最恶劣情况: M 中全是非零元素, 此时 $tu = nu*nu$,
时间复杂度为 $O(nu^2*nu)$

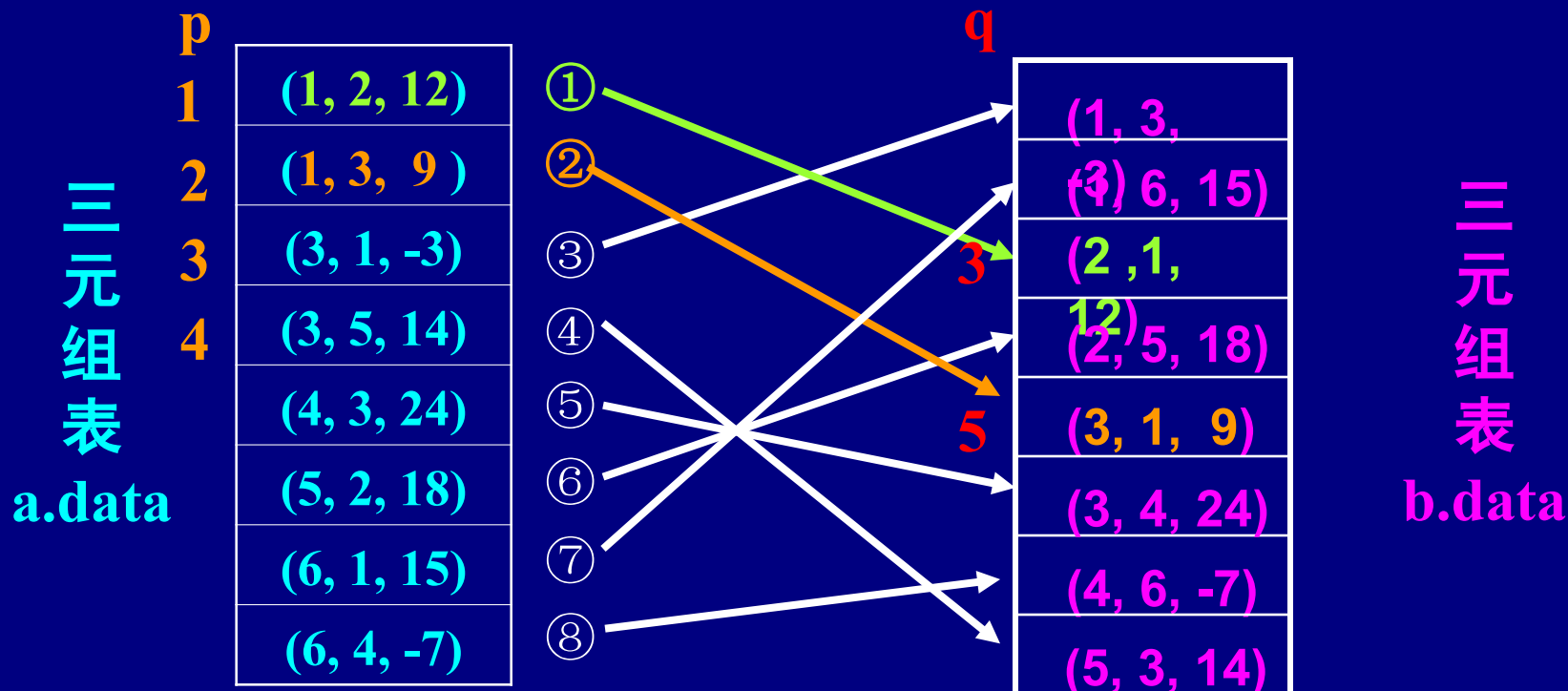
注: 若 M 中基本上是非零元素时, 即使用非压缩传统转置算法的时间复杂度也不过是 $O(nu*nu)$ (程序见教材 P99)

结论: 压缩转置算法不能滥用。

前提: 仅适用于非零元素个数很少 (即 $tu \ll nu*nu$) 的情况。

方法2 快速转置

思路：依次把 a.data 中的元素直接送入 b.data 的恰当位置上（即 M三元组的 p 指针不回溯）。



关键：怎样寻找 b.data 的“恰当”位置？

设计思路:

如果能**预知** M 矩阵**每一列** (即 T 的每一行) 的**非零元素个数**, 又能预知**第一个非零元素**在 $b.data$ 中的**位置**, 则扫描 $a.data$ 时便可以将每个元素准确定位 (因为已知若干参考点)。
注意 $a.data$ 特征: 每列首个非零元素必定先被扫描到。



技巧: 利用带辅助向量的三元组表, 它正好携带每行 (或列) 的非零元素个数 $NUM(i)$ 以及每行 (或列) 的第一个非零元素在三元组表中的位置 $POS(i)$ 等信息。

i	1	2	3	4	5	6
$NUM(i)$	2	0	2	1	1	2
$POS(i)$	1	3	3	5	6	7

请回忆:

规律: $POS(1) = 1$
 $POS(i) =$
 $POS(i-1) + NUM(i-1)$

不过我们需要的是按列生成的 M 矩阵的辅助向量。

令: M 中的列受重用 col 表示;

$\left\{ \begin{array}{l} num[col] : \text{存放 } M \text{ 中第 } col \text{ 列中非 } 0 \text{ 元素个数,} \\ cpot[col] : \text{存放 } M \text{ 中第 } col \text{ 列的第一个非 } 0 \text{ 元素的位置,} \end{array} \right.$

(即 $b.data$ 中待计算的“恰当”位置所需参考

点)

col	1	2	3	4	5	6
num[col]	2	2	2	1	1	0
cpot[col]	1	3	5	7	8	8

$$M = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

规律: $cpot(1) = 1$

$cpot[col] = cpot[col-1] + num[col-1]$

讨论: 按列优先的辅助向量求出后, 下一步该如何操作?
由 $a.data$ 中每个元素的列信息, 即可直接查出 $b.data$ 中的重要参考点之位置, 进而可确定当前元素之位置!

快速转置算法描述:

```
Status FastTransposeSMatrix ( TSMatirx M, TSMatirx &T )
{
    //M用顺序存储表示, 求 M的转置矩阵 T
    T.mu = M.nu ; T .nu = M.mu ; T.tu = M.tu ;
    if ( T.tu ) {
        for(col = 1; col <=M.nu; col++) num[col] =0; // 先统计每列非零元素个数
        for( i = 1; i <=M.tu; i ++ ) {col =M.data[ i ] .j ; ++num [col] ;}
        cpos[ 1 ] =1; // 再生成每列首元位置辅助向量
        for(col = 2; col <=M.nu; col++) cpos[col] =cpos[col-1]+num [col-1] ;
        for( p =1; p <=M.tu ; p ++ ) //p指向 a. data , 循环次数为非 0元素总个数
        { col =M.data[ p ] .j ; q =cpos [ col ]; // 查辅助向量表得 q , 即 T中位置
            T.data[q].i = M.data[p]. j;
            T.data[q].j = M.data[p]. i;
            T.data[q]. value = M.data[p]. value;
            ++ cpos[col] ; // 重要语句! 修改向量表中列坐标值, 供
                          // 同一列下一非零元素定位之用!
        } //for
    } //if
    return OK;
} //FastTranposeSMatrix;
```


快速转置算法的效率分析:

1. 与常规算法相比, 附加了生成辅助向量表的工作。增开了2个长度为列长的数组 ($\text{num}[]$ 和 $\text{cpos}[]$) 。
2. 从时间上, 此算法用了4个并列的单循环, 而且其中前3个单循环都是用来产生辅助向量表的。

for($\text{col} = 1; \text{col} \leq \text{M.nu}; \text{col}++$) 循环次数 = nu ;

for($i = 1; i \leq \text{M.tu}; i++$) 循环次数 = tu ;

for($\text{col} = 2; \text{col} \leq \text{M.nu}; \text{col}++$) 循环次数 = nu ;

for($p = 1; p \leq \text{M.tu}; p++$) 循环次数 = tu ;

该算法的时间复杂度 = $(\text{nu} * 2) + (\text{tu} * 2) = O(\text{nu} + \text{tu})$

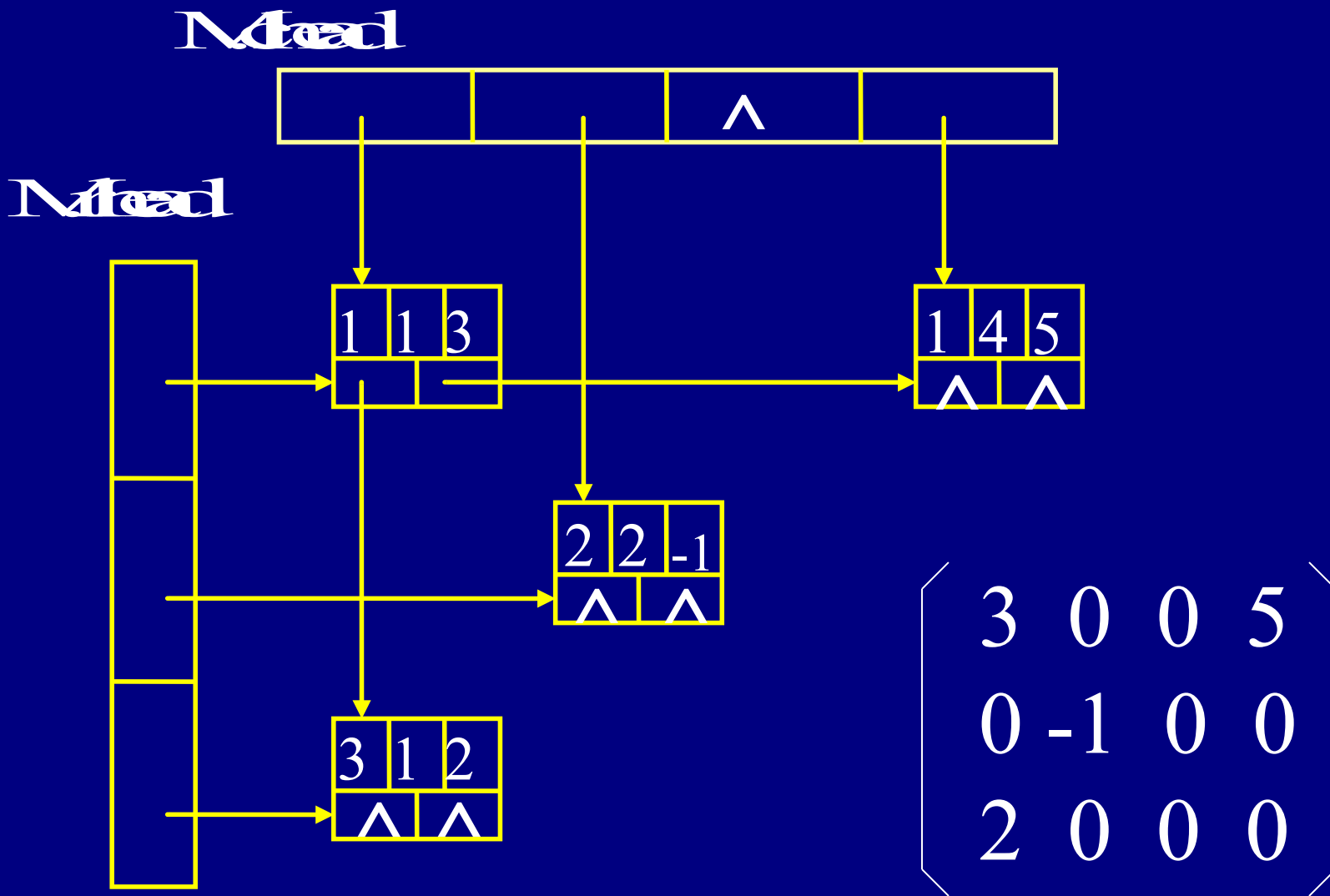
讨论: 最恶劣情况是 $\text{tu} = \text{nu} * \text{nu}$ (即矩阵中全部是非零元素), 而此时的时间复杂度也只是 $O(\text{nu} * \text{nu})$, 并未超过传统转置算法的时间复杂度。

小结: 传统转置: $O(\text{nu} * \text{nu})$ 压缩转置: $O(\text{nu} * \text{tu})$

压缩快速转置: $O(\text{nu} + \text{tu})$ ——牺牲空间效率换时间效率。

稀疏矩阵相乘的算法见教材 P101-103

三、十字链表

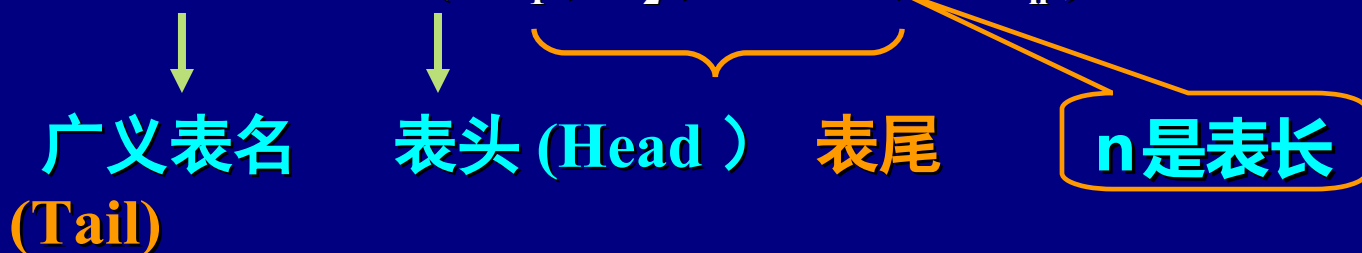


5.4 广义表的定义

1、定义：

广义表是线性表的推广，也称为列表（lists）

记为： $LS = (a_1, a_2, \dots, a_n)$



在广义表中约定：

- ① 第一个元素是表头，而其余元素组成的表称为表尾；
- ② 用小写字母表示原子类型，用大写字母表示列表。

讨论：广义表与线性表的区别和联系？

广义表中元素既可以是原子类型，也可以是列表；
当每个元素都为原子且类型相同时，就是线性表。

广义表是**递归**定义的**线性结构**,

$$LS = (\alpha_1, \alpha_2, \cdots, \alpha_n)$$

其中: α_i 或为原子 或为广义表

例如: $A = ()$

$$F = (d, (e))$$

$$D = ((a,(b,c)), F)$$

$$C = (A, D, F)$$

$$B = (a, B) = (a, (a, (a, \cdots,)))$$



广义表 $LS = (\alpha_1, \alpha_2, \dots, \alpha_n)$ 的结构特点：

- 1) 广义表中的数据元素有相对**次序**；
- 2) 广义表的**长度**定义为最外层包含元素个数；
- 3) 广义表的**深度**定义为所括弧的重数；
注意：“原子”的深度为 **0**；
“空表”的深度为 **1**。
- 4) 广义表可以**共享**；
- 5) 广义表可以是一个**递归**的表；
递归表的深度是无穷值，长度是有限值。

6) 任何一个**非空广义表** $LS = (\alpha_1, \alpha_2, \dots, \alpha_n)$
 均可分解为
 表头 $Head(LS) = \alpha_1$ 和
 表尾 $Tail(LS) = (\alpha_2, \dots, \alpha_n)$ 两部分

例如： $D = (E, F) = ((a, (b, c)) , F)$

$Head(\mathbf{D}) = \mathbf{E}$ $Tail(\mathbf{D}) = (\mathbf{F})$

$Head(\mathbf{E}) = \mathbf{a}$ $Tail(\mathbf{E}) = ((\mathbf{b}, \mathbf{c}))$

$Head(((\mathbf{b}, \mathbf{c}))) = (\mathbf{b}, \mathbf{c})$ $Tail(((\mathbf{b}, \mathbf{c}))) = ()$

$Head((\mathbf{b}, \mathbf{c})) = \mathbf{b}$ $Tail((\mathbf{b}, \mathbf{c})) = (\mathbf{c})$

$Head((\mathbf{c})) = \mathbf{c}$ $Tail((\mathbf{c})) = ()$

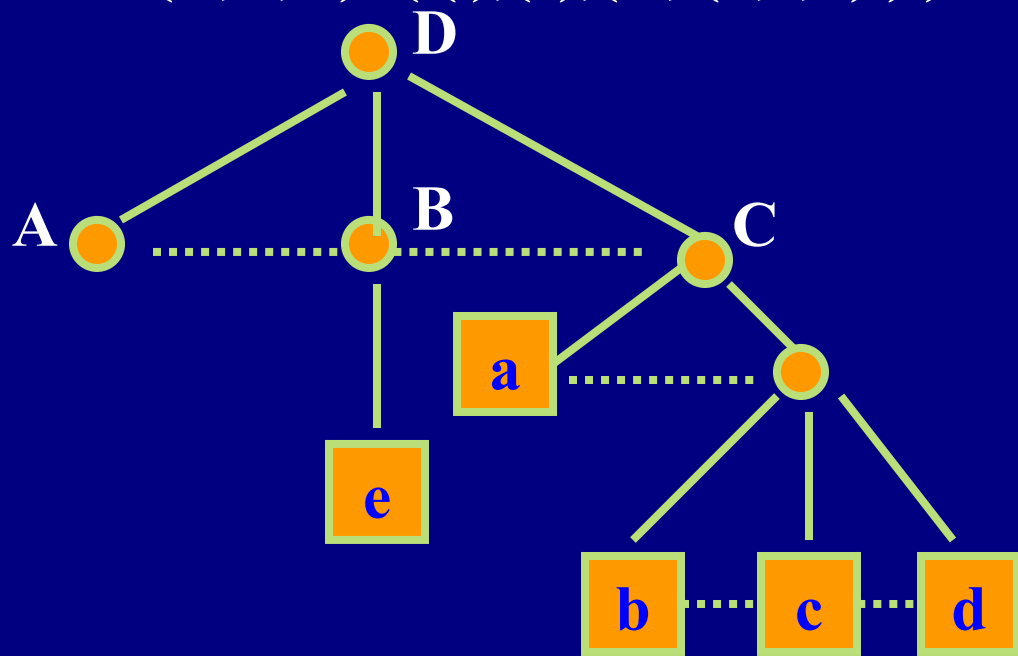
例 1 : 求下列广义表的长度。

- 1) $A = ()$ $n=0$, 因为 A 是空表
- 2) $B = (e)$ $n=1$, 表中元素 e 是原子
- 3) $C = (a, (b, c, d))$ $n=2$, a 为原子, (b,c,d) 为子表
 $n=3$, 3 个元素都是子表
- 4) $D = (A, B, C)$ $n=2$, a 为原子, E 为子表
- 5) $E = (a, E)$
- $D = (A, B, C) = ((), (e), (a, (b, c, d)))$, 共享表

$E = (a, E) = (a, (a, E)) = (a, (a, (a, \dots)))$, E 为递归表

例 2：试用图形表示下列广义表。
 (设 e 代表原子, ● 代表子表)

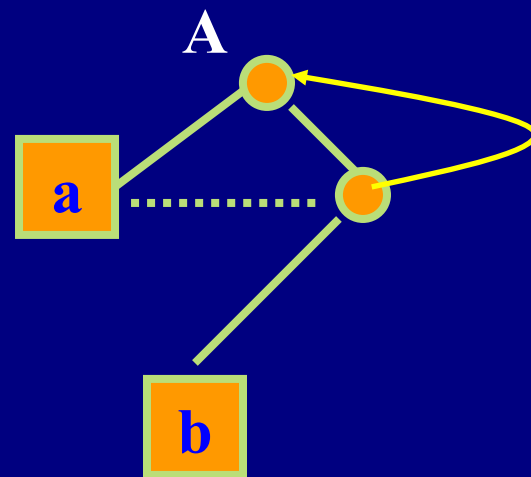
① $D=(A,B,C)=((),(e),(a, (b,c,d)))$



① 的长度为 3，深度为 3

深度 = 括号的重数 = ● 结点的层数

② $A=(a, (b, A))$



② 的长度为 2，深度为 ∞

广义表是一个**多层次的线性结构**

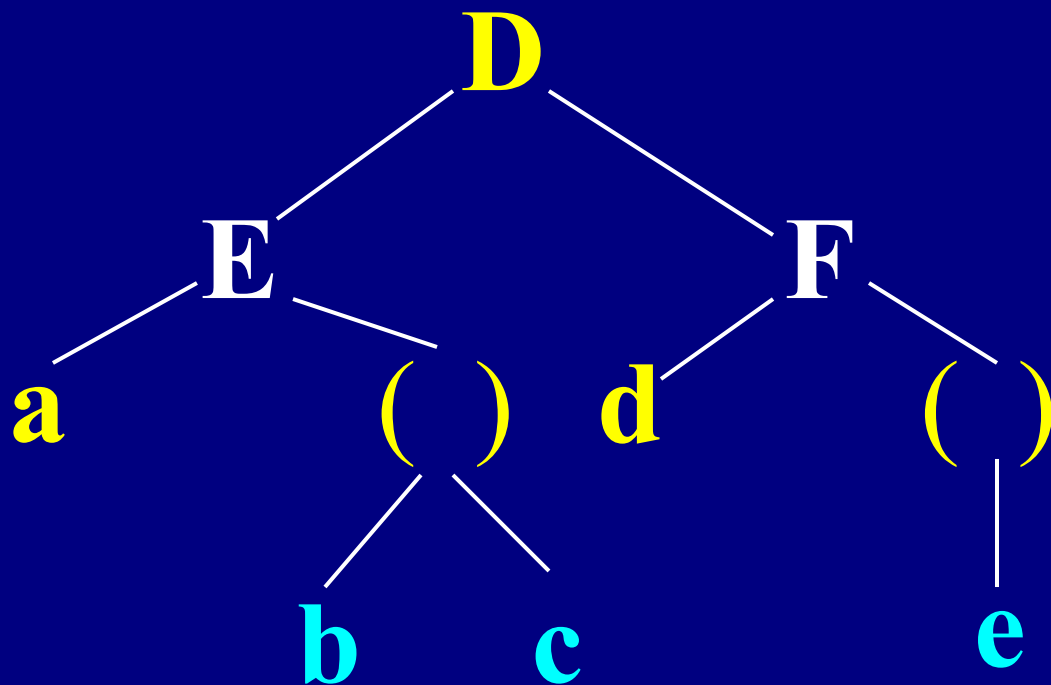
例如：

$$D=(E, F)$$

其中：

$$E=(a, (b, c))$$

$$F=(d, (e))$$



5.4 广义表的类型定义

ADT Glist {

数据对象: $D = \{e_i \mid i=1,2,\dots,n; n \geq 0;$
 $e_i \in \text{AtomSet} \text{ 或 } e_i \in \text{GList},$
 $\text{AtomSet} \text{ 为某个数据对象}$ }

数据关系:

$\text{LR} = \{ \langle e_{i-1}, e_i \rangle \mid e_{i-1}, e_i \in D, 2 \leq i \leq n \}$

基本操作:

} ADT Glist

- **结构的创建和销毁**

InitGLList(&L); DestroyGLList(&L);
CreateGLList(&L, S); CopyGLList(&T, L);

- **状态函数**

GListLength(L); GListDepth(L);
GListEmpty(L); GetHead(L); GetTail(L);

- **插入和删除操作**

InsertFirst_GL(&L, e);
DeleteFirst_GL(&L, &e);

- **遍历**

Traverse_GL(L, Visit());

广义表的抽象数据类型定义见教材 P107-108

介绍两种特殊的基本操作：

GetHead(L) —— **取表头** (可能是原子或列表);

GetTail(L) —— **取表尾** (一定是列表) 。

例：求下列广义表操作的结果（严题集 5.10②）

1. GetTail 【 (b, k, p, h) 】 = (k, p, h)

_____ ; (c,d)

2. GetHead 【 ((a,b), (c,d)) 】 = ((c,d))

_____ ;

3. GetTail 【 ((a,b), (c,d)) 】 = (b)

5. GetTail 【 ; (e) 】 = ()

(a,b)

4. GetTail 【 GetHead 【 ((a,b), (c,d)) 】 】 = _____

6. GetHead 【 (()) 】 = ()

_____ .

7. GetTail 【 (()) 】 = _____

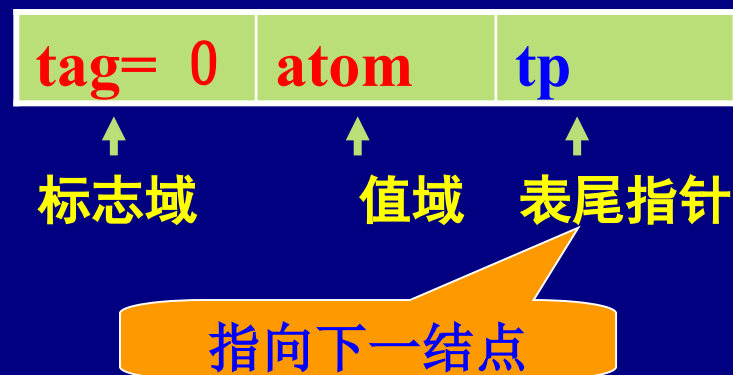
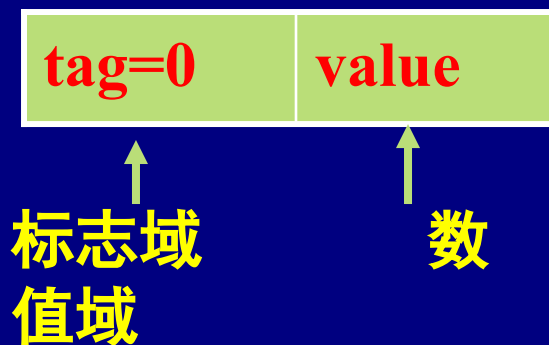
5.5 广义表的存储结构

由于广义表的元素可以是不同结构（原子或列表），难以用顺序存储结构表示，**通常用链式结构**，每个**元素**用一个结点表示。

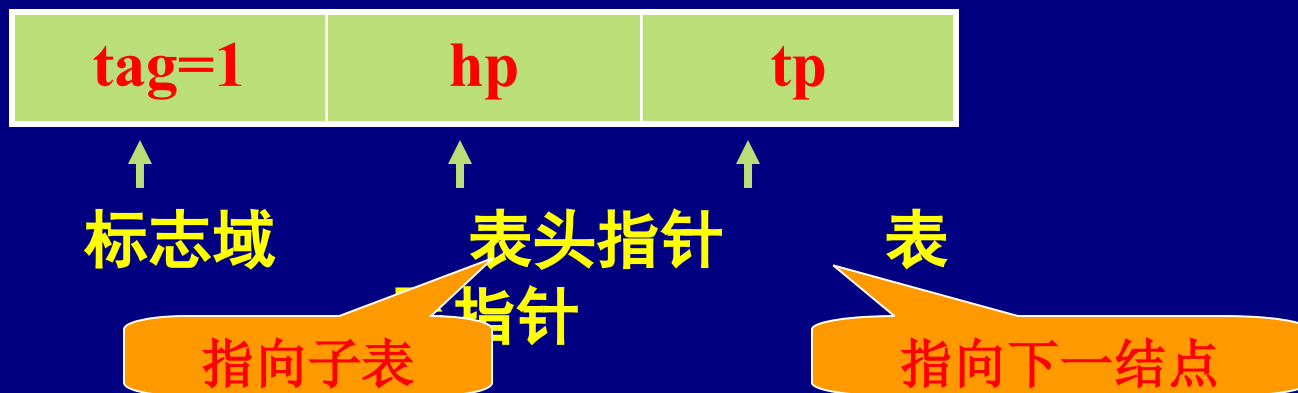
注意：列表的“元素”还可以是列表，所以结点可能有两种形式
原子结点：表示原子，可设 2 个域或 3 个域，依习惯而选。

法 1：标志域，数值域

法 2：标志域、值域、表尾指针



2. 表结点：表示列表，若表不空，则可分解为表头和表尾，用3个域表示：**标志域**，**表头指针**，**表尾指针**。

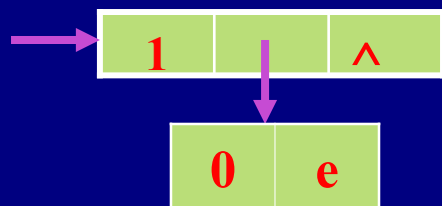


例: ① $A = ()$

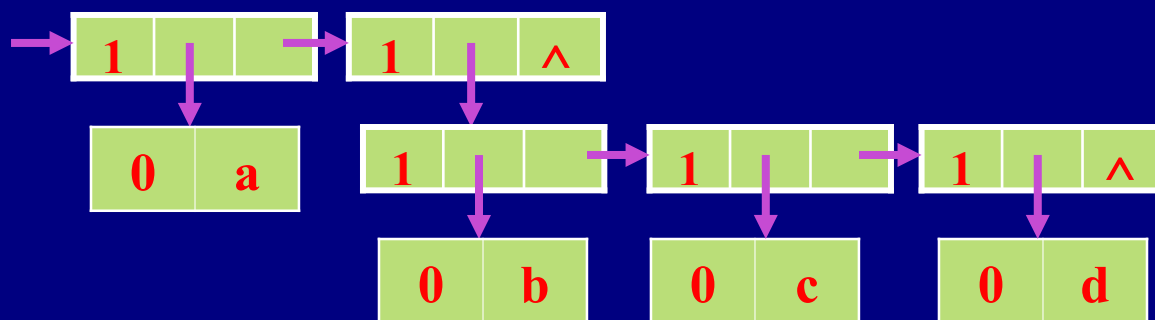
A=NULL



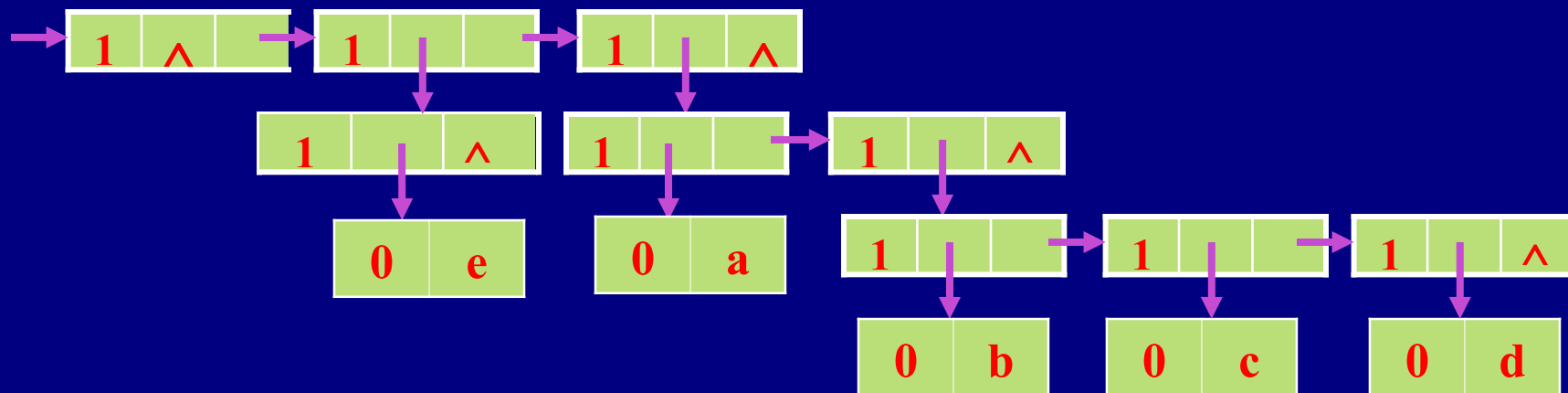
② $\mathbf{B} = (\mathbf{e})$



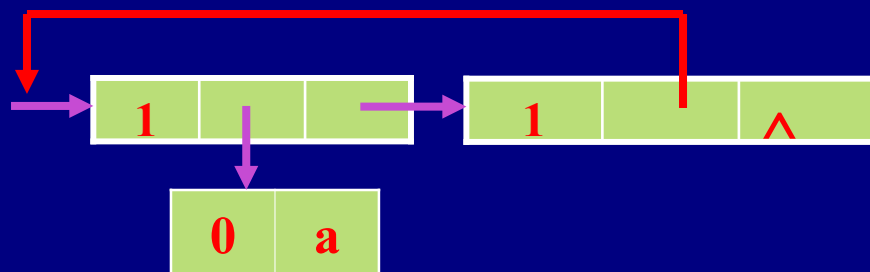
③ $\mathbf{C} = (\mathbf{a}, (\mathbf{b}, \mathbf{c}, \mathbf{d}))$



④ $D = (A, B, C) = ((), (e), (a, (b, c, d)))$



⑤ $E = (a, E)$



(参见教材 P109 图)

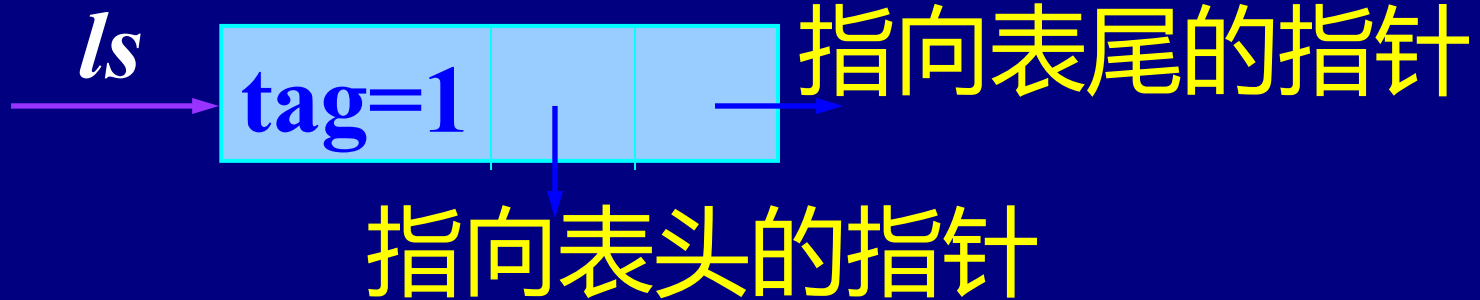
构造存储结构的两种分析方法：

1) 表头、表尾分析法：

空表

$ls = NIL$

非空表



若表头为原子，则为



否则，依次类推。

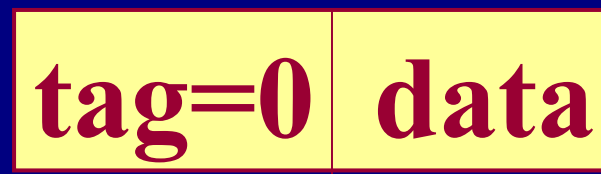
广义表的表示方法

通常采用头、尾指针的链表结构

表结点：

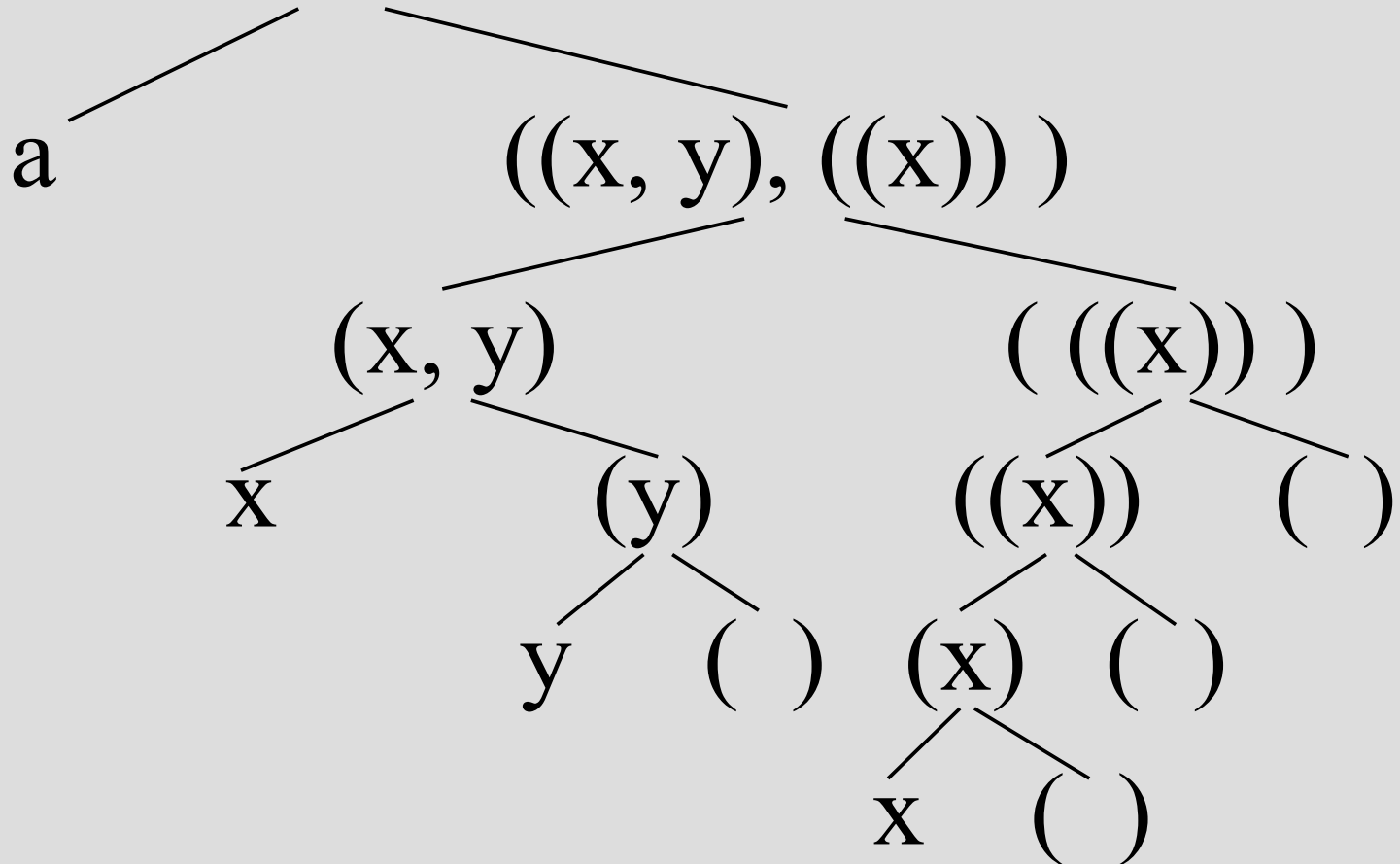


原子结点：

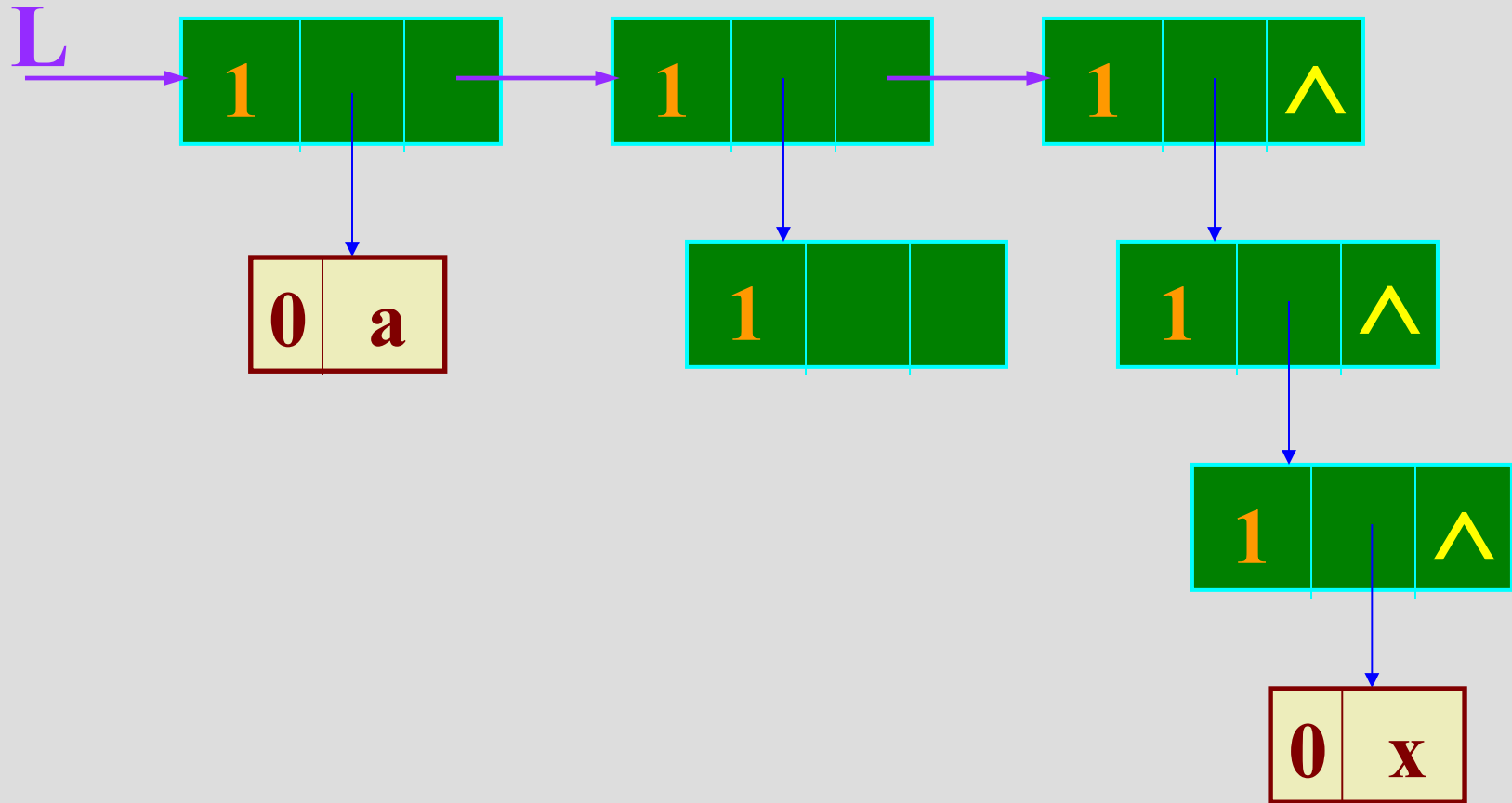


例如:

$L = (a, (x, y), ((x)))$



$$\mathbf{L} = (\mathbf{a}, (\mathbf{x}, \mathbf{y}), ((\mathbf{x})))$$

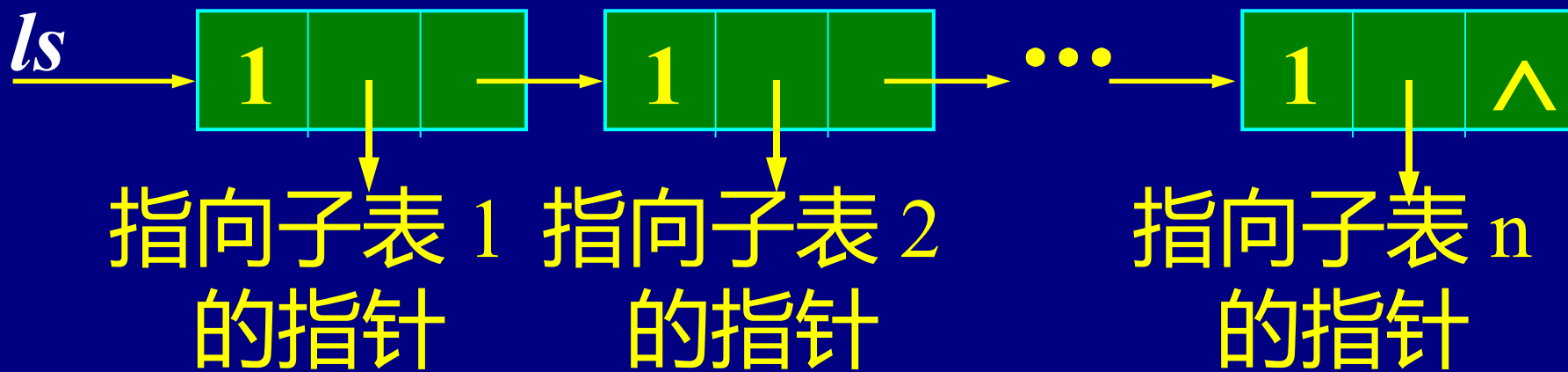


2) 子表分析法：

空表

$ls = NIL$

非空表

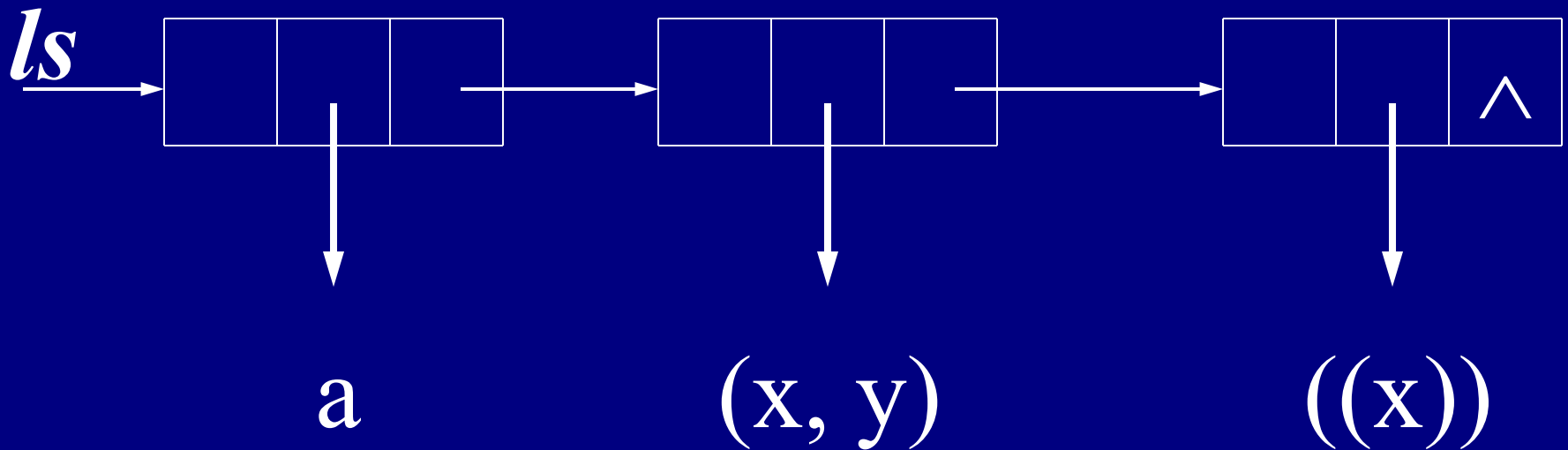


若子表为原子，则为

tag=0 data

否则，依次类推。

例如： $LS = (a, (x, y), ((x)))$



广义表从结构上可以分解成

广义表 = 表头 + 表尾

或者

广义表 = 子表 1 + 子表 2 + $\cdots$ + 子表 l

因此常利用分治法求解之。

算法设计中的关键问题是，**如何将 l 个子问题的解组合成原问题的解。**

广义表的头尾链表存储表示:

```
typedef enum {ATOM, LIST} ElemTag;
```

// ATOM==0: 原子, LIST==1: 子表

```
typedef struct GLNode {
```

```
    ElemTag tag; // 标志域
```

```
    union{
```

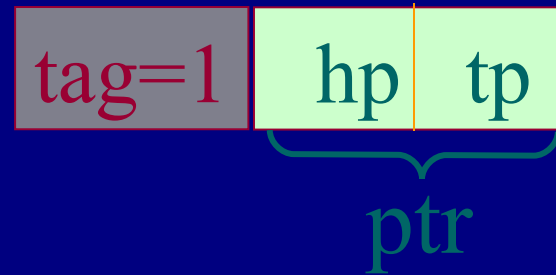
```
        AtomType atom; // 原子结点的数据域
```

```
        struct {struct GLNode *hp, *tp;} ptr;
```

```
    };
```

```
} *GList
```

表结点



1. 了解数组的两种存储表示方法，并掌握数组在以行为主的存储结构中的地址计算方法。
2. 掌握对特殊矩阵进行压缩存储时的下标变换公式。
3. 了解稀疏矩阵的两类压缩存储方法的特点和适用范围，领会以三元组表示稀疏矩阵时进行矩阵运算采用的处理方法。
4. 掌握广义表的结构特点及其存储表示方法，读者可根据自己的习惯熟练掌握任意一种结构的链表，学会对非空广义表进行分解的两种分析方法：即可将一个非空广义表分解为表头和表尾两部分或者分解为 n 个子表。